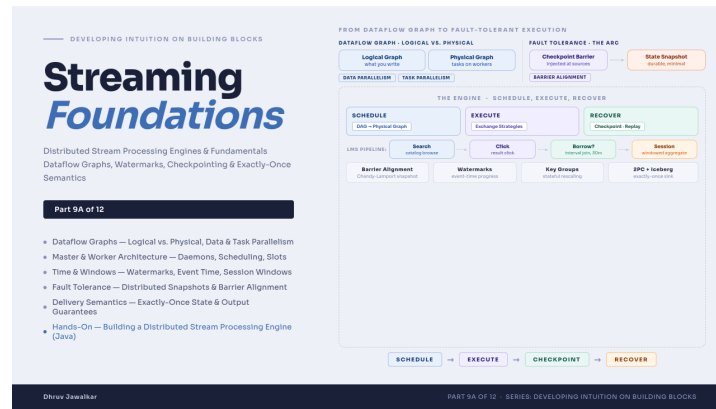




Developing Intuition on Building Blocks — for Systems Design



Dhruv Jawalkar 
Senior Software Engineer | Distributed Systems & Cloud
Infrastructure | Backend & Data Platforms | AI Infrastructure |...



September 15, 2026

Part 9A of 12: Stream Processing Fundamentals & Building an Engine

Outline:

1. Introduction & Bridge from Part 8
2. What Streaming Actually Costs
3. Dataflow Programming
4. Anatomy of an Engine
5. Moving Tuples Between Tasks
6. Time
7. Windows
8. Joining Streams
9. State
10. Fault Tolerance
11. Delivery Semantics End to End
12. Getting Comfortable with Reprocessing
13. Operating a Streaming Job
14. Keeping Derived Systems Current
15. Event-Driven Microservices vs. Stream Processors
16. The LMS Through-Line & Hands-On Project
17. Takeaways
18. Deep Dives
19. References

1. Introduction & Bridge from Part 8

7C's DeepForestMedia pipeline is a good pipeline — good enough that its account managers get exactly what was designed for them: **a daily report, refreshed on schedule**, queryable at interactive speed the moment it lands. It's also, structurally, a pipeline that can never answer the question one of those account managers actually has open on their screen right now: *did the creative I swapped an hour ago move the CTR?*

Oozie's coordinator triggers the dedup → sessionize → aggregate chain **once a day**, and only after confirming the previous day's events are fully landed. Impala sitting on top doesn't change what it's fast *at* — it queries whatever Hive's last completed run produced. The account manager isn't waiting on a slow query engine. They're waiting on a pipeline that was never built to **answer today's question with today's data**, no matter how quickly it answers yesterday's.

That gap — not "batch is slow" but "batch answers yesterday's version of today's question" — is where Part 9 starts. Two threads from Part 8 converge on it directly. 8B closed with a promise: the machinery that keeps a search index, a recommendation model, or a geo-index of moving drivers actually **current**, rather than periodically refreshed. 8A's near-real-time refresh interval was the closest thing to an answer available at the time, and it was a compromise — poll more often, still not continuous. Part 9 is where that compromise gets replaced with an actual architecture: not a search index refreshed every few seconds, but a search index *updated by a continuous process the moment the data underneath it changes*.

Getting there takes three articles. **9A** — this one — builds the fundamentals: how a distributed stream processing engine represents your computation, runs it across a cluster, tracks time, holds state, and survives failure, illustrated by building a working version of one. **9B** takes those fundamentals into the frameworks that actually run in production — Storm, Flink, Spark Structured Streaming, Kafka Streams — and what each one trades for what. **9C** goes broader: streaming SQL, streaming databases, managed cloud services, and a decision framework for choosing among all of it.

The running example for 9A's hands-on project stays inside the LMS, but shifts domain: not lending records this time, but the LMS's own **catalog-browse clickstream** — a member searching, clicking a result, and (sometimes) borrowing, all within a bounded window. Same shape of problem as DeepForestMedia's, small enough to build end to end, concrete enough that every mechanism in this article has somewhere real to land.

2. What Streaming Actually Costs

The tempting way to frame this is "streaming trades throughput for latency" — lower latency, in exchange for processing fewer records per second. It's a clean sentence and it's not true. The Flink paper shares its own benchmark on a 30-machine, 120-core cluster: 20ms p99 latency at 1.5 million events/sec, climbing to over **80 million events/sec** at a 50ms buffer timeout — on the *same* cluster, by turning one dial. Flink and Spark Structured Streaming are not low-throughput systems bought at the price of speed — that benchmark settles it. What streaming actually costs you shows up elsewhere:

- **Per-record overhead vs. amortized efficiency.** A batch job pays its setup cost once and spreads it across billions of records. A stream processor pays scheduling, serialization, and coordination overhead

continuously — batching (Section 5) is precisely how streaming engines claw that overhead back, not proof that "real" streaming means one record at a time.

- **State maintenance is now a permanent cost.** A batch job has no state between runs — it recomputes from the source every time. A streaming job's state has to live somewhere, be checkpointed, and be rescaled without loss (Section 9) — a genuinely new operational surface that batch never had to budget for.
- **Reprocessing is a design problem, not a rerun.** Rerunning a batch job means rerunning it. Reprocessing a streaming job — because of a bug fix, a schema change, or a new feature — means replaying a log against evolving state without corrupting or duplicating results (Section 12), which is a much harder promise to keep.
- **Correctness complexity moves earlier.** A batch job's input is finite and closed by the time it runs; "the answer" is unambiguous. A stream is never closed — out-of-order arrivals, late events, and "correct as of when" (Section 6) are questions every streaming pipeline has to answer explicitly, where a batch pipeline could quietly ignore them.

Dimension	Batch ETL	Stream Processing
Unit of work	Whole dataset, re-scanned each run	Per-record or micro-batch, continuously
Overhead model	Amortized over the whole run	Paid continuously; batching mitigates it
State	Recomputed from scratch each run	Maintained and checkpointed indefinitely
Reprocessing	Rerun the job	Replay against live state — a design problem
Operational profile	Runs, finishes, releases resources	Runs forever, whether or not there's traffic

Comparing Batch vs Streaming approach to ETL

This is also why "just make the batch job run more often" was never going to close 7C's gap. Running Oozie's coordinator hourly instead of daily doesn't give the account manager a live answer — it gives them a slightly less stale one, at the cost of re-scanning the same growing dataset every hour. The honest reason to reach for streaming isn't raw speed. It's that state and time stop being something the *reporting layer* has to work around, and become something the *processing engine* is built to manage directly. Section 3 starts on exactly that: how a stream processor represents the computation that does the managing.

3. Dataflow Programming

3.1 Logical vs. Physical Graphs

Every stream processing framework asks you to describe *what* your computation does, then separately works out *how* to run it across a cluster. Those are two different graphs.

The **logical dataflow graph** is the one you write: a small set of **operators** — say, filter the catalog-browse stream for well-formed events, extract the member ID and catalog item ID, key by member, then count clicks in a window — connected by the data dependencies between them. **It's a DAG** because a stream processing computation genuinely has no cycles in this sense: data flows from sources, through a fixed sequence of transformations, to sinks, and nothing loops back on itself.

or transformations, to sinks, and nothing loops back on itself.

The **physical dataflow graph** is what the engine actually runs. Each logical operator becomes one or more **tasks** — parallel copies of that operator's logic, each working on a slice of the data — scheduled onto specific machines. A logical graph with four operators might compile into a physical graph with twelve tasks spread across three **worker processes**, and the same logical program can compile to a different physical graph entirely depending on how much parallelism you ask for.

This distinction is why "the DAG" gets used loosely to mean two different things in casual conversation, and why it's worth separating cleanly before Section 4: the logical graph is what the master schedules from; the physical graph is what it actually places on workers.

3.2 Data Parallelism and Task Parallelism

There are two independent ways to get parallelism out of this graph, and it's easy to conflate them.

Data parallelism partitions the *input*: several tasks of the *same* operator, each processing a different subset of the data. This is the parallelism that lets a single count-per-member aggregation scale — ten parallel instances of the same counting logic, each responsible for a slice of the member-ID keyspace, rather than one instance drowning in the full stream.

Task parallelism runs *different* operators concurrently on the same or different data. A filter task and a downstream aggregation task can run at the same time on different machines, each doing different work, which is what lets a cluster's compute resources stay busy across an entire pipeline rather than idling while one stage runs at a time.

A production job uses both simultaneously — data parallelism to scale each operator out, task parallelism to keep the whole pipeline's stages running concurrently rather than as isolated phases.

3.3 Data Serialization: Crossing the Boundary

Task chaining's whole benefit, coming up next, is avoiding a cost — which means it's worth naming that cost first.

A record only stays a plain in-memory object as long as it never leaves the thread that created it. The moment it has to cross to another thread, another process, or another machine, it has to become bytes: **serialized** on the sending side, shipped across whatever boundary separates the two tasks, and **deserialized** back into an object on the receiving side. In a distributed stream processing engine, this happens constantly — **every key-based shuffle, every checkpoint write, every byte of state persisted to a state store** is a serialization event.

How expensive that is depends entirely on whether the engine knows the *shape* of the data ahead of time. A generic, reflection-based serializer — something that can encode arbitrary objects without being told what they look like — has to write type metadata alongside every single record and walk the object's structure at runtime to figure out how to encode it. That works for anything, but "works for anything" and "fast" are usually opposites. The alternative distributed engines converge on is generating a specific, specialized serializer for each known data type once, at the point the job is built — so encoding a member ID and a timestamp becomes a fixed, **compact binary layout** with no metadata overhead and no runtime type inspection, repeated a billion times over the life of a job. A general-purpose fallback like **Kryo** still exists for the types the engine doesn't recognize, but it's explicitly a fallback, not the default path — falling back

to it for a type that's on your job's hot path is a real, measurable performance regression, not a rounding error.

For the LMS project, this is the difference between a ClickEvent { memberId, catalogItemId, eventType, timestamp } encoding to a compact fixed layout the engine generated specifically for that shape, versus the same object silently falling onto a generic path that additionally writes out class and field metadata on every single record shuffled across the cluster.

One thread worth flagging now and picking up properly later: this same serialized representation is what gets written into state stores and into checkpoints (Sections 9 and 12) — so the serializer you're using isn't just a network-cost decision, it's also what determines whether you can safely change your data types later without breaking a job you're trying to **restart from a saved checkpoint**.

3.4 Exchange Strategies

Once a record has been serialized to bytes, something still has to decide *which* task those bytes are addressed to. That decision is the exchange strategy, and it's chosen either automatically by the engine (a keyed operation forces a key-based exchange) or explicitly by the programmer.

Strategy	What it does	Cost
Forward	Sends output directly to one specific downstream task	Cheapest — often avoids the network entirely if both tasks land on the same machine
Key-based	Partitions by a key attribute; all records with the same key go to the same task	Moderate — required whenever correctness depends on co-locating a key (e.g., counting per member)
Broadcast	Sends every record to every downstream task	Expensive — replicates data and always crosses the network
Random	Distributes records evenly, regardless of content	Moderate — used purely to balance load when no correctness constraint exists

Data Exchange Strategies post an Operator Task completing the defined computation

Key-based exchange is the one worth sitting with, because it's the one the LMS catalog-browse project depends on directly: once click events are keyed by member ID, every event for a given member is guaranteed to land on the same task instance — which is the only way a per-member session count can be computed correctly in a data-parallel operator at all. Get the exchange strategy wrong here — random instead of key-based — and the aggregation isn't slow, it's *wrong*: the same member's events scattered across tasks that never see each other's counts.

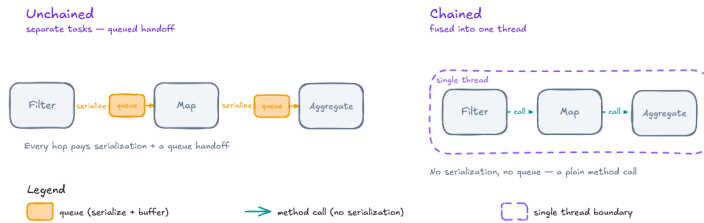
3.5 Task Chaining

Data parallelism and exchange strategies explain how work spreads *out*. Task chaining is the **optimization** that pulls adjacent work back *together* when spreading it out would only add cost.

When two or more operators are configured with the same parallelism and connected by a forward exchange, the engine can fuse them into a single task running in a single thread — a record handed from one function to the next by a plain method call, **with no serialization and no queue in between**. A filter immediately followed by a field-extraction map, both running at parallelism four and wired forward-to-forward, is exactly this case: there's no reason to pay for a buffer handoff between two steps that are always going to run on the same machine in the same order.

Task Chaining

Fusing adjacent operators into one thread skips serialization entirely



Chained vs. Unchained operator pipeline — three operators fused into one thread with method-call handoff, versus the same three operators as separate tasks communicating over buffered channels

Chaining isn't free of trade-offs, though. Because chained functions share a single thread, one slow function in the middle of a chain blocks every function scheduled after it in that same thread — there's no separate thread free to keep the rest of the pipeline moving while it catches up. That's exactly why engines let you break a chain deliberately: giving an expensive function its own dedicated task trades away the method-call savings in exchange for scheduling it independently, so it can't stall its neighbors.

Either way, this is the concrete mechanism behind Section 2's claim that streaming's per-record overhead is a real, addressable cost: Section 3.3 named what that overhead actually is, and chaining is the engine refusing to pay it for a handoff that never needed to leave the thread in the first place.

Section 4 picks up where this leaves off: the physical graph exists, tasks are defined, records know how to become bytes and find their way to the right task — now who decides which machine each task actually runs on, and who keeps track once it's running.

4. Anatomy of an Engine

4.1 Master and Worker

Strip away the branding and every distributed stream processing engine needs the same two roles: something that knows the whole job and decides where its pieces run, and something that actually runs them. Storm and Flink name these differently, but the shape underneath is the same — with one genuine architectural difference worth calling out rather than smoothing over.

Role	Apache Storm	Apache Flink
Master daemon	Nimbus — schedules topologies, tracks cluster state, handles <code>activate</code> / <code>deactivate</code> / <code>rebalance</code> / <code>kill</code>	JobManager — converts the logical graph into a physical ExecutionGraph , requests resources, coordinates checkpoints
Worker daemon	Supervisor — one per worker node; starts and restarts worker processes on that machine	TaskManager — the worker process itself; registers its slots with a ResourceManager
Resource negotiation	Implicit — Nimbus assigns tasks straight to vacant worker-process slots	Explicit — a dedicated ResourceManager brokers slot requests between the JobManager and TaskManagers, and can provision new

		TaskManager containers on demand
Submission entry point	Nimbus itself	A Dispatcher — a long-lived REST endpoint that spins up a fresh JobManager per submitted application
Coordination service	ZooKeeper — holds the state needed to coordinate Nimbus and Supervisors	ZooKeeper — leader election and durable pointers, in high-availability mode

The Low Level and High Level Components needed for a Distributed Stream Processing Engine

Flink's split of "master" into three separate daemons (JobManager, ResourceManager, Dispatcher) isn't a different idea from Storm's single Nimbus — it's the same control-center responsibility, decomposed so that resource brokering can plug into YARN, Kubernetes, or Mesos without the job-tracking logic caring which one.

4.2 What a Slot Actually Is — Where the Analogy Breaks

Both frameworks use the word "slot," and here the parallel genuinely stops. In Storm, a slot corresponds 1:1 to a worker process — an entire JVM. Two used slots means two JVMs running somewhere in the cluster. It's coarse-grained: reserving a slot means reserving a whole process.

In Flink, a slot is finer-grained and structured differently: it's one **vertical slice through the entire pipeline** — one task from every operator, not one whole worker process. A TaskManager offering two slots isn't running two independent things; it's running two complete slices of the job side by side, each slice pulling one parallel task from every operator in the graph. A pipeline with operators at parallelism four needs at minimum four slots to run at all, however many TaskManagers those four slots are spread across.

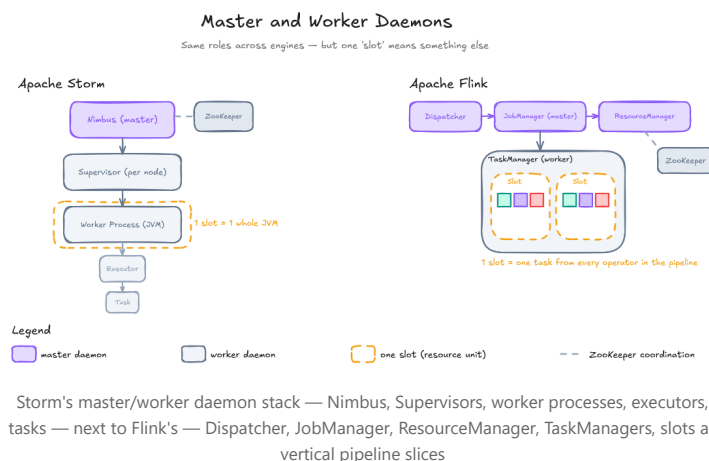


Diagram link

This directly explains why task chaining (Section 3.5) works the way it does: chained tasks have to be colocated in the same slot to share a thread and skip serialization — chaining is only possible *because* a slot already guarantees one task per operator sitting together, ready to be fused.

4.3 Job Submission, End to End

Putting Sections 3 and 4 together: a client packages the job and hands it to the entry point — Nimbus directly in Storm, a Dispatcher in Flink. The

to the entry point — Nimbus directly in Storm, a dispatcher in Flink. The master converts the submitted logical graph into a physical graph (Section 3.1), then requests the processing units it needs — vacant worker-process slots in Storm, negotiated TaskManager slots in Flink — and assigns physical tasks to whatever it's granted. From that point on, tasks exchange data using exactly the exchange strategies from Section 3.4, whether that data crosses a network or, inside a chained slot, never leaves a thread at all.

4.4 Tracking a Running Job

Once a job is running, the master needs a live picture of it — which tasks are healthy, which have died, how far each one has progressed. Both frameworks keep this picture current through worker daemons reporting status back on a heartbeat: a worker node's absence is noticed by its silence, not announced by it. Storm additionally exposes this state through the Storm UI; the equivalent operational tooling and what to actually watch day to day is Section 13's territory, not this one — here it's enough to know the master always has an answer to "is this task still alive."

4.5 When the Master Itself Is the Failure

This is where the two designs genuinely diverge, and it's worth stating plainly rather than papering over.

Storm's Nimbus is explicitly not a single point of failure for work already running. Storm's own fail-fast reasoning covers this directly: if a worker node dies, its Supervisor restarts it; if Nimbus itself dies, it's running under external supervision (daemontools, monit) and simply restarts. Critically, while Nimbus is down, Supervisors and worker processes *keep processing* — what's lost isn't execution, it's the ability to reassign failed tasks to healthy machines or deploy new topologies, until Nimbus comes back.

Flink's JobManager is a harder problem: it genuinely is a single point of failure for the application it controls. The JobManager holds the live coordination state — including pointers to the last completed checkpoint — and a streaming application simply cannot continue if its JobManager disappears, full stop. Flink's answer is a high-availability mode, and it leans on the exact coordination pattern Part 6 established and Part 8A revisited for SolrCloud: **ZooKeeper for leader election**, plus metadata (the job graph, checkpoint pointers) **written to durable remote storage** rather than kept only in the JobManager's memory. A standby JobManager, elected leader on failure, fetches those pointers from ZooKeeper and takes over.

Neither design is simply better — Storm trades a more centralized dependency for a simpler recovery story on healthy paths; Flink trades a harder failure mode for a cleaner, checkpoint-consistent recovery once HA is configured. What actually happens to in-flight tasks and state during that recovery — not just who becomes the new leader — is Section 10's job.

Our own engine, when we build it in Section 15, has to make this exact call honestly rather than gloss over it: a master process holding job metadata is only as reliable as the story for what happens the moment it dies, and that story is a design decision, not an afterthought.

5. Moving Tuples Between Tasks

5.1 Same-Process Transport: The Disruptor Queue

When two tasks share a worker process, handing a record from one to the other is exactly Part 1's producer-consumer pattern, built concrete: one thread produces, another consumes, and a queue sits between them so neither has to wait on the other in lockstep. Storm's implementation of that queue is worth naming specifically, because it's a real, widely-used piece of infrastructure in its own right: the **LMAX Disruptor**, a lock-free ring buffer library built for exactly this kind of high-throughput inter-thread handoff, and Storm uses it for every one of its internal executor queues.

Here's the tuple's actual journey inside a single JVM. A spout's main thread runs the user code that emits a tuple and places it on that executor's **outgoing disruptor queue**. A second, dedicated **send thread** drains that queue and hands the tuple directly to the next executor's **incoming disruptor queue** — and if both executors live in the same worker process, this handoff is about as cheap as inter-thread communication gets: no network, and critically, no serialization, because the tuple never has to leave the process's memory. This is precisely the case Section 3.3 pointed at: local transfer is what happens when two tasks *are* co-located but weren't chained into the same thread — cheaper than a network hop, but not as cheap as chaining's plain method call.

5.2 Crossing a Process or a Machine

The moment two tasks don't share a JVM, the tuple has to become bytes and survive a real network hop — the cost Section 3.3 introduced in the abstract becomes concrete here.

Storm's path: the sending executor's send thread serializes the tuple — Storm looks for a registered Kryo serializer first, falling back to slower standard Java serialization if none exists, exactly the fast-path-vs-fallback trade-off from 3.3 — then publishes it to a *second* disruptor queue, this one scoped to the entire worker process rather than a single executor: the **worker's outgoing transfer queue**. A dedicated worker-level thread drains that queue and ships the tuple over a persistent TCP connection (via Netty) to the destination JVM, where a receive thread accepts it, deserializes it, and routes it into the correct destination executor's incoming queue.

Flink's path takes a different shape at the same layer: rather than shipping tuples individually, a TaskManager collects outgoing records into fixed-size buffers (32KB by default) before sending them, batching for throughput over a permanent TCP connection maintained per pair of TaskManagers. Worth sitting with: the Flink book itself flags that this buffering means Flink's "continuous" streaming transport is, underneath, a form of micro-batching too — a detail that'll matter directly once 9B draws Spark's micro-batch model as a sharp contrast to Flink's "continuous" one. One real cost worth naming now: with a shuffle or broadcast exchange, each sending task needs a dedicated buffer for *every* receiving task — buffer requirements scale quadratically with task count, a genuine tuning concern at high parallelism that Section 13 picks back up.

5.3 Backpressure: Two Different Answers to the Same Problem

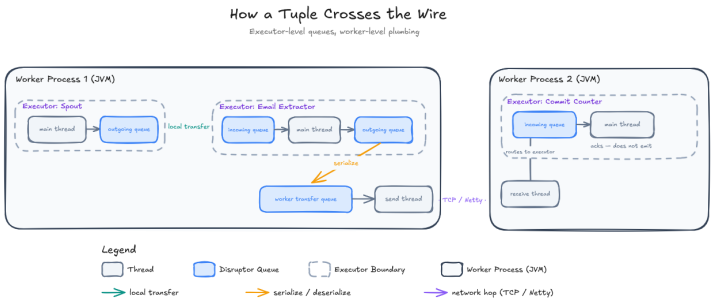
A continuous pipeline has no natural pause point the way a batch job does between runs — so something has to stop a fast task from burying a slow one in unprocessed work. Storm and Flink solve this at genuinely different points in the pipeline.

	Storm: Max Spout Pending	Flink: Credit-Based Flow Control
Mechanism	A static cap, set ahead of time, on	A live, per-connection negotiation — a

	how many tuples a single spout task may have in flight without having finished processing	receiving task grants a sender exactly as much reserved buffer space ("credit") as it currently has room for
When it acts	Proactively — you calculate a safe ingest cap before the pipeline ever runs	Reactively and continuously — a receiver falling behind simply grants less credit
Where it applies	Only at the spout, throttling ingest at the source	At every connection in the graph, so backpressure propagates upstream hop by hop on its own
Tuning burden	On the programmer, worked out from parallelism math ahead of time	Largely automatic, though buffer pool sizing still needs attention at scale

Controlling the rate of flow between Operator Tasks - Backpressure implementations in Storm and Flink

Storm's own metaphor for max spout pending is a dam: cap the ingest rate before the flood starts, and everything downstream stays within a bound you calculated by hand. Flink's credit system instead lets every hop continuously renegotiate its own safe rate, reacting automatically to a skewed key or a temporarily slow task without anyone precomputing anything. Neither is strictly better — Storm's is simpler to reason about and adds no live coordination overhead; Flink's reacts to load changes Storm's static cap can't see coming, at the cost of running that negotiation machinery continuously. This exact contrast is one of the rows 9B's four-engines comparison table will need to fill in for Spark and Kafka Streams too.



A tuple's full journey — Spout's Main Thread → Outgoing Disruptor Queue → Send Thread → local transfer (same JVM) or serialize → Worker Transfer Queue → TCP/Netty → Destination Worker → Executor's Incoming Disruptor Queue

[Diagram link](#)

6. Time

6.1 Two Clocks, Not One

Every record in the catalog-browse clickstream carries two different timestamps, and they're rarely the same value. **Event time** is when the member actually clicked — the moment captured on their device. **Processing time** is when our engine happens to see that record — which, for a member on a spotty mobile connection, might be minutes later, **arriving out of order** relative to clicks that happened after it but reached us first.

Processing time is tempting because it's simple: use the local machine clock, no coordination needed, lowest possible latency. It's also the less trustworthy choice for anything that has to be *correct* rather than just fast — a job replayed from a checkpoint using processing time will group

events into different windows than the original run did, because the clock it's reading has nothing to do with the data itself. Event time is reproducible for exactly that reason: the timestamp travels with the record, so replaying the same data always produces the same windows, regardless of when the replay happens to run. (A third option, ingestion time — the moment a record enters the engine — sits between the two: more stable than processing time, cheaper than tracking true event time end to end. Worth knowing it exists; the project sticks with event time throughout, since it's the one the rest of this article's mechanisms are actually built around.)

The catch with event time is the one this whole section exists to solve: if you're going to group and window on when something *happened*, you need some way of knowing when you've seen everything that's going to happen for a given window — in a system where "everything" is, by definition, never fully arrived.

6.2 Watermarks: A Global Progress Measure

A **watermark** is a special control record, injected into the stream alongside ordinary data, carrying a single timestamp t — and asserting a claim: no event with an event-time earlier than t is still coming on this partition. It's not a promise about what *has* arrived; it's a promise about what *won't* arrive any earlier than this from here on.

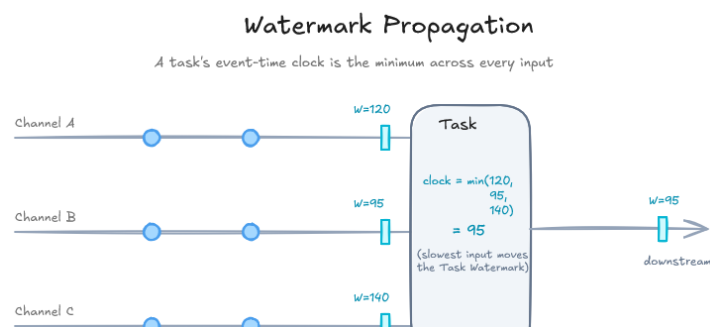
That claim is what lets a window operator (Section 7) ever declare itself done. Without watermarks, a count-clicks-per-member window would have no principled way to know whether it's seen every click in that window or is just seeing a lull — it could only guess, or wait forever. A watermark passing the end of a window is the engine's actual permission slip to compute and emit that window's result.

6.3 Propagating Watermarks Through a DAG

A source doesn't just watermark itself and stop — that timestamp has to travel through every downstream task, and a task with more than one input has to combine several incoming watermarks into one honest answer.

The rule is deliberately conservative: a **task tracks a separate watermark per input partition**, and its own event-time clock is always the **minimum** across all of them. Only when that minimum advances does the task fire any timers waiting on it and re-emit its own, newly-advanced watermark downstream. This is the only safe choice — a task can't claim "I've seen everything up to t " if even one of its inputs hasn't gotten that far yet, no matter how far ahead the others are.

This is exactly why the project's stream-stream join in Section 8 — matching a search-result click against a later borrow — can't fire early just because the borrow-events partition is running fast. Its output watermark is pinned to whichever of the two input streams is behind, by constructio





Only once the minimum advances does the task fire due timers and forward its own watermark

Legend

 watermark

 ordinary record

A task with three input partitions and one output partition, tracking per-partition watermarks, computing the minimum as its event-time clock, and broadcasting the advanced watermark to its downstream connection

6.4 The Idle-Partition Bug

The minimum-of-all-inputs rule has a sharp edge, and it's one of the more common ways a real streaming job goes quietly wrong: if even one partition stops producing data *and* stops producing watermarks — a low-traffic catalog category overnight, a partition whose upstream producer briefly stalls — the minimum across all inputs simply stops advancing, because that one silent partition never contributes a higher number. Every downstream task waiting on that clock freezes: windows stop firing, timers stop triggering, state keeps accumulating — not because anything crashed, but because one quiet source is holding the whole event-time clock hostage.

The fix has to be explicit, not automatic: a source that knows it has genuinely gone quiet can declare itself **idle**, and the propagation algorithm then excludes that partition from the minimum calculation entirely until it produces data again. Without this, a job can look completely healthy on every dashboard — CPU fine, no errors, records flowing on every other partition — while quietly accumulating unbounded state and unfired windows behind one partition nobody thought to watch.

6.5 The Latency/Completeness Dial

Where watermarks actually get generated — at the source, or via a function that periodically inspects each record's timestamp — is a choice with a real trade-off baked in. Generate watermarks aggressively (assume little skew, advance the clock quickly) and windows fire sooner, at the cost of excluding events that arrive later than expected. Generate them conservatively (assume a longer tail of stragglers) and you catch more late data, at the cost of every window result arriving later.

There's no universally correct setting here — it's a dial, not a fact, and where you set it depends entirely on how tolerant your use case is of a wrong-but-fast answer versus a right-but-slow one. Section 7's triggers and allowed lateness are the mechanism that lets you have some of both: fire an early, approximate result when the watermark first passes, and revise it if a genuinely late event shows up after.

7. Windows

7.1 What: Window Types

A window is how an otherwise-endless **stream gets cut into finite, computable pieces** — a **bucket of events**, with rules for which events go into which bucket. Three shapes cover nearly everything the project needs:

Tumbling windows assign events into fixed-size, **non-overlapping buckets** — every event belongs to exactly one window. Counting catalog-item clicks in fixed 5-minute buckets is a tumbling window: window 1 closes cleanly before window 2 begins, with no event double-counted across the boundary.

Sliding windows are fixed-size but *overlapping* — a new window starts at a fixed interval shorter than the window's length, so a single event can belong to several windows at once. A rolling 10-minute view of search activity per member, recomputed every minute, is a sliding window: at any moment there are ten overlapping windows in flight, each one minute apart from its neighbors.

Session windows don't have a fixed size at all — a session is a run of events with no more than some **gap of inactivity** between them, closed only once that gap is observed. This is the one the project actually leans on: a member's catalog-browse session — search, click, maybe another search, maybe a borrow — with the window's boundary defined entirely by a 15-minute gap of silence, not a clock tick. Two members active at the same time get sessions of completely different lengths, because the data itself decides where each one ends.

All three types can also run **partitioned by key** — one independent set of windows per member, per catalog item, whatever the key is — which is the case the project actually needs, since a global session window across every member at once would be meaningless.

7.2 When: Triggers and the Window Lifecycle

Deciding *what* goes in a window is only half the problem — something also has to decide *when* a window is done and ready to be evaluated, and that's the trigger's job, kept deliberately separate from the window assigner.

A window is born the moment its assigner routes the first element into it — there's no such thing as an empty window waiting around in advance. From that point it holds three things: its **content** (either the raw buffered elements, or a running aggregated value — Section 7.3 is the difference), an **end timestamp** marking when it becomes eligible for deletion, and whatever timers and state its trigger has registered for itself. Every assigner ships with a sensible default trigger — an event-time tumbling window's default trigger simply **fires once the watermark** (Section 6) **passes the window's end** — but the trigger can be replaced with custom logic for cases like "fire early every time 100 events accumulate, in addition to firing at the normal boundary."

When the trigger fires, the configured window function runs and emits a result; when the window's end timestamp is finally passed (plus any allowed lateness, next section), the operator deletes the window's content outright. This is exactly why state can silently grow unbounded if a trigger is written to fire but never actually purge — the window operator only clears content it knows the trigger is finished with.

7.3 How: Accumulation

Two elements land in the same window in very different ways depending on whether the window operator is configured to aggregate *incrementally* or hold onto the *raw* elements.

With an **incremental aggregation function** — a running sum or count — each new element is folded into the current result the instant it arrives, and the window's entire content is just that one running value. This is cheap: a session's click count never needs to store the actual clicks, just a number that goes up by one. With a **full window function** instead, the operator has no choice but to buffer every raw element until the trigger fires, because the computation genuinely needs to see the whole set at once — "the three most-viewed catalog items in this session" can't be computed incrementally the way a plain count can.

computed incrementally the way a plain count can.

The project's own operators split cleanly along this line: the windowed click-count aggregation is incremental and cheap; anything that needs the full session's shape (which Section 15 will use to decide the join in Section 8) has to buffer and pay the memory cost that comes with it.

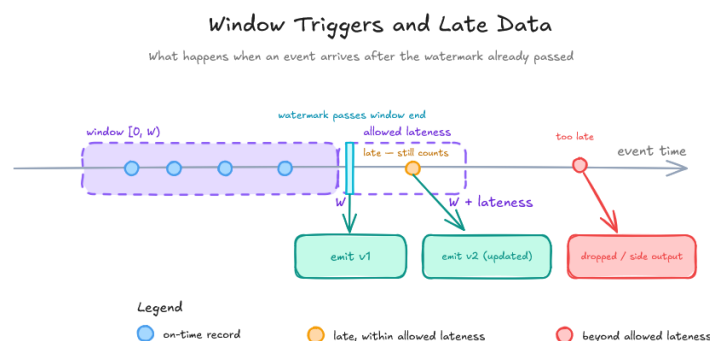
7.4 Handling the Stragglers: Allowed Lateness and Late Data

Section 6.5 named the latency/completeness dial; this is where a job actually has to act on wherever that dial gets set. A **late element** is one that **arrives after the watermark has already passed the end of the window** it belongs to — the trigger has already fired, and by default, the window's content is already gone.

The default behavior is to simply drop it — the safest choice when a little inaccuracy is cheaper than the complexity of doing anything else. Two other options exist when dropping isn't acceptable:

- **Redirect to a side output** — route late elements to a separate output stream rather than discarding them, so they can be logged, audited, or folded back in later through a periodic backfill process rather than corrupting the live result.
- **Extend the window's life with allowed lateness** — configure the operator to keep a window's full state alive for a defined grace period *past* its normal end, so a late element arriving within that grace period is treated like any on-time element, and the operator emits an *updated* result rather than dropping the late data outright.

That second option comes with a real downstream obligation worth flagging now rather than later: a sink receiving updated results for a window it already emitted once needs to be able to handle that update — typically by overwriting the earlier result rather than appending a duplicate. That's not a footnote; it's the exact reason Section 11's sink discussion cares whether a destination supports upsert-style writes at all, and it's precisely the kind of correctness question an Iceberg sink (Section 11, Section 15) has to answer honestly rather than assume away.



A window's trigger timeline — on-time elements accumulating, the watermark passing the window boundary and firing the trigger, a late element arriving within the allowed-lateness grace period, and the resulting updated emission.

8. Joining Streams

8.1 Why Both Sides Need to Be Buffered

A stream-stream join is a fundamentally different problem from the join

A stream-stream join is a fundamentally different problem from the joins Part 2 covered, because *neither* side is sitting still. In a relational join, one table is fully materialized before the query runs — every row is already there to be matched against. In a stream-stream join, both inputs are still arriving, and — because of the out-of-order delivery Section 6 established as a fact of life — the engine has no guarantee about which side's matching event shows up first. A borrow event could physically reach the join operator before the click event that triggered it has finished being processed there, purely because of network or partition timing, even though logically the click always happens first.

The consequence is unavoidable: a stream-stream join operator has to **hold state on both sides, buffering events** from each input while it waits for a potential match on the other. This is the first thing that makes stream-stream joins meaningfully more expensive than the stream-table joins in Section 8.5 — there, only one side needs to be held in state at all.

8.2 The Interval Join

The project's actual join — did a member borrow the item within 30 minutes of clicking its search result? — is a textbook case for an **interval join**: two streams sharing a key (member + catalog item), matched when their timestamps fall within a bounded, asymmetric interval of each other. A click at time t matches any borrow between t and $t + 30 \text{ minutes}$ — the bound doesn't have to be symmetric, and doesn't have to include the past at all here, since a borrow logically can't precede its own click.

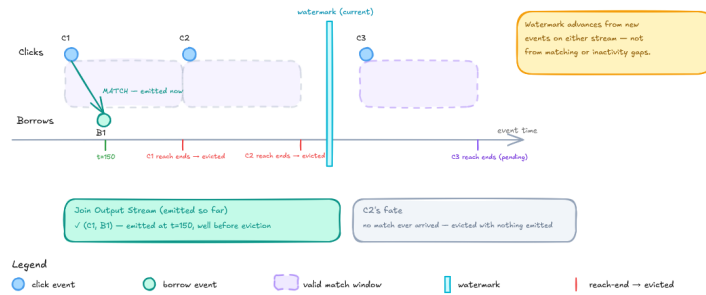
That bound is exactly what makes buffering tractable: the operator only needs to retain click events whose 30-minute window hasn't closed yet, and borrow events recent enough that some not-yet-expired click could still match them. Once the current watermark moves past a given click's 30-minute deadline, that click can never produce a match — its buffered state can be safely dropped.

8.3 Watermark-Driven Eviction — and the Unbounded Join

This is where Section 6.3's watermark rule stops being an abstract correctness guarantee and becomes a genuine **memory-management mechanism**: the watermark isn't just what tells a window when it's safe to fire — it's what tells a join when it's safe to *forget*. An interval join can only evict a buffered event once the watermark proves no future match is still possible for it.

That dependency cuts both ways, and it's worth stating plainly: because the watermark is pinned to the *slower* of the two input streams (Section 6.3's minimum-of-all-inputs rule), an interval join between a fast-moving stream and a slow, laggy one buffers far more than either stream alone would suggest — the fast stream's events pile up waiting for a watermark that's being held back entirely by the slow one.

The failure mode this all guards against is worth naming directly: a join written with **no time bound at all** — match this click to *any* borrow, whenever it happens to arrive — gives the engine no principled moment to ever declare a buffered event safe to discard, because a match could theoretically arrive years later. **State grows** without limit, silently, for as long as the job runs. This is precisely the state-growth risk Section 9 covers more generally — the interval join is simply the clearest place to see *why* an explicit bound isn't a stylistic choice, it's the only thing that makes eviction possible at all.



An interval join's buffering region — click events get buffered in its state store for the valid-match duration (30 mins) until a matching borrow event arrives and a join output is emitted, new events advance the watermark, evicting expired events

[Diagram link](#)

8.4 The Window Join — a Different Trade-off

The alternative is a **window join**: rather than a per-event interval, both streams are assigned into the *same* windows (Section 7), and every element from stream A is matched against every element from stream B that landed in the same window — a cross-product, evaluated once the window's trigger fires.

This trades the interval join's flexibility for windowing's clean, predictable grouping — genuinely the right shape when what you actually want is "everything that happened together in this bucket," not "within N minutes of this specific event." But it comes with a sharp, easy-to-miss edge: two events one second apart, but on opposite sides of a window boundary, simply never join — even though an interval join covering the same one-second gap would have caught them without hesitation. Worth knowing this exists as an option; the project sticks with the interval join, since an artificial window boundary has no natural meaning for a single member's click-to-borrow relationship.

8.5 Stream-Table Enrichment and Temporal Joins

Most joins in production aren't stream-stream at all — they're **stream-table**: enriching a fast-moving event with slower-changing reference data. Attaching each click event's catalog-item title and category is exactly this case. Only one side needs state here (the table), which is already a lighter problem than Section 8.1's.

The naive version — look up whatever the table currently says — has a quiet correctness problem: it's non-deterministic on replay. If a catalog item's category gets recategorized next week, reprocessing today's clicks would enrich them with next week's category, producing a different result than the original run did, purely because the table moved on underneath it. A **temporal (versioned) table join** is the fix: rather than joining against the table's current state, join against the table's state *as of the event's own timestamp* — which requires the table itself to be **tracked as a changelog of updates** over time, not a single mutable snapshot.

That changelog framing is not a coincidence — it's the same stream-table duality thread that CDC tools like Debezium produce and that Kafka Streams' KTable abstraction is built entirely around, both of which get their full treatment in 9B and Section 14. For now, the takeaway that matters: a lookup join that silently reads "whatever the table says right now" is a correctness bug waiting for a reprocessing run to expose it.

9. State

9.1 Operator State vs. Keyed State

Flink draws a hard line between two scopes of state, and the difference is about *who's allowed to see it*.

Operator state is scoped to a single parallel task — every record that task happens to process can read and update it, but no other task of the same or any other operator can touch it. A source task tracking its own current Kafka offset is the canonical example: that offset belongs to *this* task's slice of the partitions, and no other task has any business seeing it.

Keyed state is scoped to a key value, not a task — Flink maintains one state instance per key and automatically scopes every access to whatever key the current record carries. This only works at all because of a guarantee Section 3.4 already established: key-based exchange routes every record for a given key to the same task, every time. Keyed state doesn't add that guarantee — it *depends* on it. A per-member session object, holding a running click count and the timestamp of the member's last activity, is exactly this shape: every event for member 4471 always reaches the one task responsible for member 4471's state, because the exchange strategy already promised as much back in Section 3.

9.2 State Primitives

Keyed state comes in a small set of shapes, chosen to match what the computation actually needs rather than defaulting to one generic container:

- **Value state** — a single value per key. The project's per-member session object (click count, last-seen timestamp) is a value state.
- **List state** — a list of entries per key, for when a computation needs to retain a sequence rather than collapse it.
- **Map state** — a key-value map per key, for when the state itself is naturally keyed a second time (per-catalog-item counts *within* a member's session, say).

Operator state has its own smaller set (list, union list, broadcast state), mostly relevant to source connectors and configuration that needs to be identical across every parallel task — not central to the project's own operators, so left at that.

9.3 State Backends: Memory vs. RocksDB

Declaring state is one thing; something still has to decide how it's actually stored, accessed, and kept safe if a task dies. That's a **state backend**, and the choice is a genuine trade-off, not a default to accept blindly.

	In-memory (heap) backend	RocksDB backend
Access speed	Fast — plain JVM objects	Slower — every read/write serializes to and from an embedded store
Size ceiling	Bounded by JVM heap	Bounded by local disk, far larger
Checkpoint behavior	Full state snapshot each time	Supports incremental checkpoints

Comparing maintaining State In-Memory to RocksDB backend with worker local/ remote storage persistence

RocksDB showing up here is worth naming directly rather than treating as a fresh idea: this is the same LSM-tree structure Part 4b covered for Cassandra and 7C/7D revisited for HBase and ClickHouse, doing a genuinely different job here — not a shared, cluster-wide database, but a small, **embedded, per-task store** holding only that one task's slice of keyed state, **written to local disk** specifically so a task's state isn't capped by however much heap the JVM happens to have. The engine chooses RocksDB for the same underlying reason Part 4b's systems did — **its write path is what makes checkpointing large state affordable** — and that mechanism, incremental checkpointing via immutable **Sorted Strings Table** files, is exactly what DD1 exists to unpack properly.

9.4 State TTL and Preventing Leaks

This is the failure mode that actually shows up in production, and it's worth walking through concretely because the fix isn't automatic — Flink deliberately does not clean up state on its own, since doing so could silently change an operator's semantics.

The classic trigger is an **evolving key domain**: a key that's only active for a while and then simply stops appearing. A per-member session key is precisely this — a member is active for a burst of clicks, then goes quiet, potentially forever as far as this job is concerned. A stateful operator has no way to know in advance that a given event is the *last* one it will ever see for that key — so left alone, it keeps that key's state around forever, on the off chance one more event shows up. Multiply that by every member who's ever used the LMS catalog, and state grows without bound for exactly the same underlying reason Section 8.3's unbounded join did: no principled moment ever arrives to say "this can safely be forgotten."

The fix is the same shape both times: give the operator an explicit signal for when it's safe to evict. Two ways to do it — **attach a TTL** to the state value itself, so it expires automatically after a period of inactivity, or register an explicit **timer** for some point in the future, with a callback that clears the state if no new event arrived before the timer fired. And here the project's own session windows (Section 7.1) hand this problem its answer for free: the same 15-minute inactivity gap that closes a session window is the exact signal that should also clear any per-member state held outside the window itself — one concept, reused rather than reinvented.

9.5 Rescaling: Key Groups

One more practical question worth answering here rather than leaving implicit: what actually happens to keyed state when a job's parallelism changes — scaling from four tasks to eight because traffic grew? Redistributing individual keys one at a time would mean tracking every single key's new location, which gets expensive fast at real scale.

The fix is **key groups**: keys are partitioned into a fixed, larger number of key groups by a stable hashing function, independent of the job's current parallelism. Rescaling then redistributes whole key groups to tasks rather than individual keys — a coarser, cheaper unit to move, and one that doesn't need to change no matter how many times the job's parallelism itself changes later.

Section 10 picks up directly from here: state is what actually gets checkpointed, and everything in this section — what's held, where it lives, how large it can grow — is exactly what a consistent snapshot has to capture without ever stopping the stream to take it.

10. Fault Tolerance

10.1 The Naive Approach, and Why It Fails

The simplest possible way to **checkpoint a running job** is also the one every latency-sensitive system has to reject outright: pause everything, write out every task's state, resume. It's trivially consistent — nothing moves while the snapshot is taken — and it's also a global stop-the-world pause, repeated on every single checkpoint, for the entire life of a job that's meant to run for months. For anything resembling the project's LMS clickstream, that's not a tuning problem to optimize away later; it's disqualifying on its own.

What's needed instead is a way to take a **consistent snapshot** — one where every operator's state genuinely corresponds to the same logical moment — without ever stopping the stream to get it. That's a distributed snapshot problem, and it has a known, decades-old answer.

10.2 Chandy-Lamport, Applied: Checkpoint Barriers

Flink's checkpointing is built directly on the **Chandy-Lamport algorithm** for distributed snapshots, adapted to the shape of a dataflow graph rather than an arbitrary network of processes. The adaptation hinges on a single new kind of record: a **checkpoint barrier** — injected into the stream by source tasks, carrying a checkpoint ID, and behaving like Section 6's watermarks in one crucial respect: it can never be overtaken by the records around it. Everything before the barrier belongs to this checkpoint; everything after belongs to the next one.

A checkpoint starts at the master: **the JobManager sends a checkpoint-ID message** to every source task. Each source pauses just long enough to checkpoint its own local state, then **broadcasts the barrier** on every one of its outgoing partitions before resuming normal processing. From that point, the barrier is on its own, traveling through the graph exactly like any other record — just one that nothing is allowed to pass.

10.3 Barrier Alignment

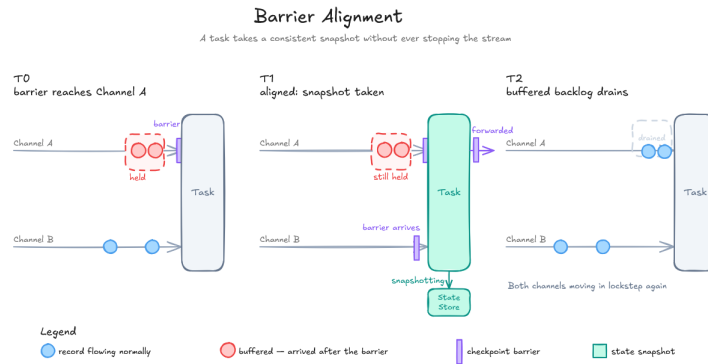
This is the part worth slowing down for, since it's the mechanism that makes the whole thing correct rather than just clever.

A task with only one input simply checkpoints its state the moment the barrier arrives and forwards it. A task with *multiple* inputs — the far more interesting and common case — has to wait until it's seen the barrier on **every** input partition before it can safely checkpoint. While it's waiting, it doesn't stall entirely: records still arriving on partitions that haven't produced their barrier yet keep being processed normally. But records arriving on a partition that *has* already sent its barrier get set aside — **buffered**, not processed — because processing them now would smuggle post-checkpoint effects into a snapshot that's supposed to represent the moment just before the barrier. This waiting period, buffering some inputs while still consuming others, is **barrier alignment**.

Once every input's barrier has arrived, the task does two things in order: it checkpoints its own state to the backend, then broadcasts the barrier onward to every downstream task — and only after that does it finally process the backlog of records it had been buffering.

The reason this is worth the ceremony, rather than just checkpointing in-flight records directly, comes down to the shape of the graph itself: because a dataflow is a DAG, there's no need to separately capture records that are mid-flight between tasks at all — the alignment phase itself

guarantees that every record's effects have already landed in *someone's* checkpointed state by the time the snapshot completes. What gets persisted is genuinely the minimum possible: operator state, and nothing else.



A multi-input task with checkpoint barriers arriving at different times on different partitions, one partition's records piling up in a buffer while the other keeps flowing, the moment of alignment and state snapshotting

[Diagram link](#)

10.4 Sinks and Checkpoint Completion

Every task in the graph — not only sinks — checkpoints its own state and **sends the JobManager an acknowledgment** the moment its own alignment finishes. Sinks perform the same alignment as any multi-input task, checkpoint their own state, and acknowledge too. The JobManager only marks the whole checkpoint **complete** once it's received an acknowledgment from every single task in the application. Until that happens, the previous completed checkpoint is still the one recovery would fall back to.

10.5 The Cost of Alignment — and Where It Meets Backpressure

Alignment isn't free, and it's worth naming exactly where the cost lands: a task can only finish aligning as fast as its *slowest* input's barrier arrives, and a backpressured channel (Section 5.3) is exactly the case where a barrier gets stuck behind a long backlog of ordinary records queued ahead of it. The two mechanisms interact directly — a pipeline already under backpressure doesn't just process slower, its checkpoints take measurably longer too, because the very channel causing the backpressure is the one alignment is waiting on.

Flink's answer, at least as this material describes it, is a deliberate trade-off available at exactly this bottleneck: for applications that can tolerate at-least-once instead of exactly-once state consistency, alignment can be configured to stop buffering entirely — every record gets processed as it arrives, barrier or no barrier — and the checkpoint is simply taken once all barriers have shown up, even if it now includes some effects that technically belong to the next checkpoint. On recovery, those records get reprocessed, which is exactly what downgrades the guarantee from exactly-once to at-least-once. (A further-evolved version of this idea — persisting the in-flight buffered records themselves as part of the checkpoint, avoiding the guarantee downgrade altogether — exists in later Flink versions under the name *unaligned checkpoints*; it's not covered in the material I've been given for this article, so I've kept this section to the trade-off the book actually describes. Worth adding properly if you come across a source covering it.)

10.6 Recovery: Task, Worker, and Master

...the recovery, idempotent, and master.

Recovering from a checkpoint is a fixed, three-step sequence, and the first step is the one that tends to surprise people: **restart the entire application** — not just the task that failed — **reset every stateful task's state to the latest completed checkpoint**, and **resume processing from there**. A single crashed task can't be recovered in isolation, because the rest of the pipeline has already moved past the checkpoint that task is being reset to; only resetting everyone to the same consistent point preserves correctness. (Some later Flink versions add finer-grained, region-scoped recovery as an optimization on top of this baseline — again, not something the material here covers in detail.)

This same restart-from-checkpoint mechanism handles both **task-level** failures and **worker-level** ones — a TaskManager dying simply means the JobManager has fewer slots available and has to acquire replacements (from the ResourceManager, if the deployment supports elastic provisioning) before the same three-step recovery can proceed. The failure's origin differs; the recovery mechanism doesn't.

Master-level failure needs a genuinely different mechanism, because the JobManager holds the very metadata that recovery depends on — Section 4.5 named this as a real single point of failure and pointed to ZooKeeper as the fix. Here's what that fix actually stores: in high-availability mode, the JobManager **writes the job graph and its metadata to durable remote storage**, and writes only a *pointer* to that location into ZooKeeper — not the data itself. As each checkpoint completes, the same pattern repeats: state handles go to remote storage, a pointer to ZooKeeper. When the JobManager dies, a standby elected through ZooKeeper's leader election never needs anything from the old process's memory at all — everything it needs to resume is a pointer away, in storage that outlived the process that wrote it.

Storm's answer to this whole section is worth a single honest note rather than a deep comparison: it doesn't take global consistent snapshots at all. Its reliability model is built on a fundamentally different mechanism — per-tuple acknowledgment and replay — which trades away the clean "reset everyone to the same barrier" guarantee for a lighter-weight, per-record accounting scheme. That mechanism gets its full treatment in 9B; the point worth taking from it here is that "fault tolerance" in stream processing isn't one idea with two implementations — it's two genuinely different architectural bets.

11. Delivery Semantics End to End

11.1 Two Different Promises: State vs. Output

Section 10 earned the application exactly-once **state consistency** — after a failure, every operator's internal state is restored to precisely what it was at the last checkpoint. It's tempting to assume that also means exactly-once *results*, and it doesn't. Recovery **replays** everything the sources produced after that last checkpoint, and the sinks that already emitted those results the first time around have no idea a replay is happening — they'll happily receive and forward the same results a second time. State consistency and **output consistency** are genuinely separate promises, and getting the first one doesn't hand you the second for free.

Getting *any* end-to-end guarantee at all has a prerequisite worth stating plainly: the **source has to be resettable** — able to rewind its read position to exactly where a checkpoint says it was, the way Kafka's offsets allow and a plain socket never could. Without that, a failure can lose data outright,

capping the whole pipeline at at-most-once no matter how good the sink is. With a resettable source, the sink is what decides whether replayed output becomes a real problem:

Source	Sink	End-to-end guarantee
Non-resettable	Any	At-most-once
Resettable	Ordinary (no special handling)	At-least-once
Resettable	Idempotent	Exactly-once, with a caveat (11.2)
Resettable	Two-phase commit	Exactly-once

The properties at Source and Sink; and how they affect pipeline End-to-end guarantees

11.2 Idempotent Sinks

A write is **idempotent** if performing it twice produces the same result as performing it once — upserting a key-value pair into a map, for instance, since the second identical upsert changes nothing that the first one didn't already set. If a sink's writes are naturally idempotent, replayed output stops being a correctness problem almost incidentally: the duplicate write just overwrites itself into the same final state.

The catch is a real one, not a technicality: this only works if replay deterministically produces the *same* keys it produced the first time — an upsert keyed by a random or time-of-write value would duplicate rather than overwrite. And there's a subtler cost worth being honest about: while a replay is actively catching back up to where the failure happened, anyone reading the sink mid-recovery can see genuinely strange intermediate states — a count that briefly drops before climbing back past its old value, because the replay is overwriting later results with earlier ones on its way through. The sink is only consistent again once the replay passes the point where the original failure occurred.

11.3 Transactional Sinks: Write-Ahead Log vs. Two-Phase Commit

When a sink's writes aren't naturally idempotent — appending to a list, say, where every write is a genuinely new event rather than an overwrite — the fix has to **control *when* a write becomes visible**, not just what it contains.

A **write-ahead-log (WAL)** sink is the general-purpose version: buffer every result in the operator's own checkpointed state, and only flush the buffer out to the real external sink once notified that the checkpoint covering those results has fully completed. It works against almost any sink, since the sink system itself doesn't need to know or cooperate — but it grows the job's state with every buffered record, and it turns steady output into a bursty flush right after each checkpoint rather than a continuous trickle.

A **two-phase commit (2PC)** sink asks more of the destination — genuine transactional support, or something close enough to fake it — and gives back a cleaner result. For each checkpoint interval, the sink opens a transaction and writes records into it as they arrive, visible to nothing outside that transaction. Only once the checkpoint is confirmed complete does the sink commit — and the commit protocol isn't a separate piece of machinery bolted on afterward, it's Section 10's checkpoint protocol *reused directly*: the checkpoint barrier arriving is the signal to open a transaction, each task's per-checkpoint acknowledgment is effectively its vote to commit, and the JobManager's checkpoint-complete notification is the actual instruction to commit. The same barrier-alignment machinery that made state consistent is, without any extra coordination layer, also running two-phase commit for the sink.

11.4 The Iceberg Sink, Concretely

This isn't abstract for the project — it's exactly how the LMS session-aggregation results land in Iceberg. Iceberg's own commit model is already built around atomic snapshot swaps: new data files can be written to storage freely, but they're invisible to any reader until a new snapshot pointer is atomically committed to the table's metadata. That's close enough to 2PC's shape that the mapping is almost direct: data files get written throughout a checkpoint interval — the "prepare" phase, present on disk but not yet part of any committed snapshot — and only once the JobManager confirms the checkpoint complete does the job commit a new Iceberg snapshot referencing those files, making them visible to every downstream reader in one atomic step.

If the job fails before that commit happens, the orphaned data files were never referenced by any snapshot in the first place — nothing needs to be rolled back, because nothing was ever visible. Recovery simply resumes from the last committed snapshot and writes the same interval's data again, this time committing cleanly.

11.5 The Multi-Table Transaction Analogy, Revisited

This is where the comparison held back since Section 2 finally lands with an actual mechanism behind it, not just a metaphor. A single event causing updates across several tables in a relational schema needs one atomic transaction wrapping all of them — a partial write, visible to some reader mid-transaction, would leave the database in a state nobody ever intended to expose. The database's answer is the same shape every time: stage every change, then commit all of it in one indivisible step, or none of it.

A checkpoint-coordinated 2PC sink is running the identical logic, just stretched across a different unit of work: the **"transaction"** spans a **checkpoint interval** instead of a single request, the barrier's arrival plays the role of the client's write, and the JobManager's checkpoint-complete notification plays the role of the commit call — but the promise is word-for-word the same one a relational database makes. Nothing becomes visible until the whole unit is ready, and if anything fails partway through, none of it is visible. Iceberg's snapshot commit is simply that same promise, implemented at the table-format layer instead of inside a database engine.

12. Getting Comfortable with Reprocessing

12.1 Kappa vs. Lambda: One Log, Replayed

The 2015 Flink paper opens with a direct complaint about how most systems handled this before: the **lambda architecture** — a fast streaming path for approximate, timely results running alongside a separate batch path for slow, accurate ones — works, but it comes at a real cost: high latency from the batch side, real **complexity** from running and coordinating two entirely different systems, and a genuine **risk of inaccuracy** from implementing the same business logic twice and hoping the two versions agree.

What the industry came to call the **Kappa architecture** is the alternative this whole article has quietly been building toward: don't maintain two pipelines at all. **Keep exactly one stream processing job**, one piece of code, running against one durable, replayable log. When you need to reprocess — fix a bug, backfill a gap, recompute history with corrected logic — you don't reach for a different system. You run the *same job* again.

log. — you don't read from a different system. You run the same job again, pointed at an earlier position in the log.

This only works because of choices this article already made deliberately, not by accident: **event time** (Section 6.1) makes replay produce the same windows regardless of when the replay happens to run — processing time never could. A **resettable, durable source** (Section 11.1) — Kafka's durably retained, replayable log, the infrastructure Part 3 already established — is what makes "point at an earlier position" a real operation rather than a wish. Reprocessing isn't a separate architecture bolted alongside streaming. It's the same pipeline, run again.

12.2 Savepoints vs. Checkpoints

Section 10 built checkpoints as an automatic, continuous safety net — periodic, silently discarded once superseded, existing for exactly one purpose: restart the *same* application, unmodified, after an unplanned failure. A **savepoint** is built from the identical underlying mechanism (the same consistent-snapshot algorithm from Section 10.2–10.3), but everything about its lifecycle and intent is different.

	Checkpoint	Savepoint
Triggered by	Automatically, on a schedule	Explicitly, by a person or external tool
Lifecycle	Auto-discarded once superseded	Kept indefinitely until deliberately removed
Intended restart target	The same application, unchanged	A <i>different but compatible</i> application

Comparing Checkpoints and Savepoints

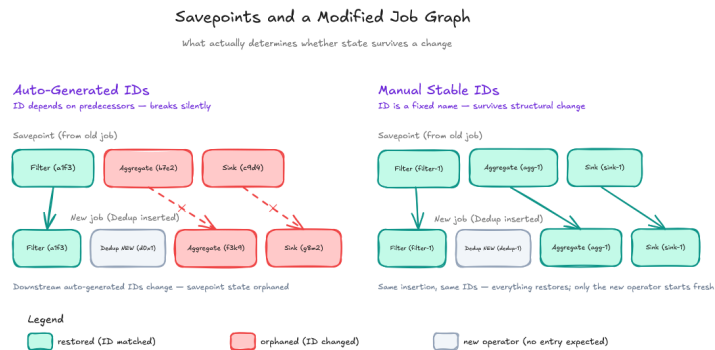
That last row is the whole point. A savepoint is meant to be the starting point for a bug-fixed version of the job, a version running with more or less parallelism, a version deployed to a different cluster or a newer engine version entirely, or simply a job you're pausing now and resuming next quarter. Section 10's checkpoint answers "how do I survive a crash." A savepoint answers "how do I deliberately change this job without losing everything it's ever learned."

12.3 State Schema Evolution and Restoring a Modified DAG

What actually makes "start a *different* application from this state" possible is a detail worth being precise about: a savepoint doesn't store state per task — it stores state per **operator**, identified by an **operator ID** and a **state name**. Because it's not tied to task count, Section 9.5's key groups can freely redistribute that state across however many tasks the restored job happens to run with. Parallelism can change freely across a savepoint restore, by design.

What can't change safely, without a specific precaution, is the operator's identity itself. By default, an operator's ID is auto-generated from a hash that includes its *predecessors* in the graph — which means adding, removing, or reordering an operator anywhere upstream silently changes the ID of every operator downstream of that change, whether or not you touched them. Restore from an old savepoint after that kind of change, and the new job simply can't find a matching entry for those operators' state — not an error, just silently orphaned state and a fresh start for the operator that was supposed to inherit it. The fix is simple and worth adopting from day one rather than discovering the hard way: **assign every stateful operator a manual, stable ID** up front, specifically so the job's structure can evolve later without state quietly falling through the cracks.

This is also exactly where Section 3.3's forward-pointer about serializers resolves. A stored session object's *shape* changing across a version upgrade — adding a field, say — only restores cleanly if the serializer that wrote it can also read an older version of that same type. Getting the serializer choice right back in Section 3 wasn't only a network-cost decision; it's the same decision that determines whether this section's promise — evolve the job, keep the state — is actually available to you late



A Savepoint's per-operator state, tagged by Operator ID, mapped onto a modified job graph
— Auto-Generated Operator IDs change on adding a new Operator breaking Savepoint restore, Manual IDs survive structural change

[Diagram link](#)

12.4 Running a New Version in Parallel

Everything above enables one more pattern worth naming explicitly, because it's the difference between trusting a change and hoping it works: take a savepoint from the currently running job, start the *new* version — bug fix, tuning change, new business logic — from that exact savepoint on a separate deployment, and run both side by side against the same live input for a validation window. Compare outputs. Watch for regressions. Only once the new version has proven itself does the old one get retired and traffic cut over for good.

This isn't a named Flink feature so much as the natural consequence of savepoints being data, not a one-way migration: nothing stops two separate jobs from being started from the same savepoint, so a **canary-style validation** that's routine practice in ordinary service deployments becomes available for a stateful streaming job too — a genuinely reassuring option for a job whose "just roll it back" story is otherwise a lot more involved than restarting a stateless web server.

Reprocessing, at every level this section covered — replaying the log to fix a bug, evolving a job's shape without losing its memory, validating a new version before trusting it — is what turns 12.1's "just replay the log" from a slogan into something you can actually do safely and repeatedly, session by session, without dread.

13. Operating a Streaming Job

13.1 Four Signals, Not Four New Concepts

Everything worth watching about a running job is a mechanism this article already built, now turned into something you observe rather than reason about in the abstract.

- **Lag** — how far behind the live edge of the source the job currently sits. For our own pipeline, that's the gap between a click actually happening and it showing up in the latest committed Iceberg snapshot (Section 11.4) — the number a product analyst waiting on today's borrow-conversion figures would feel directly, well before any internal alert fires. Because the source is Kafka, this specific signal doesn't need anything built for it: consumer group lag is a metric Kafka already exposes per partition, the same infrastructure Part 3 established.
- **Checkpoint duration and state size** — Section 10's checkpoints, watched over time rather than reasoned about once. A checkpoint that keeps taking longer, or state that keeps climbing without leveling off, is usually the first visible symptom of either Section 9.4's leak or the skew this section gets to next.
- **Backpressure status** — Section 5.3's mechanism, surfaced live, per task, as a simple signal: is this task currently blocked waiting to send, or flowing freely. A single task stuck flagged while its siblings sit idle is close to a direct pointer at exactly where a problem lives.
- **Time spent in barrier alignment** — Section 10.5's cost, made observable: how many bytes got buffered waiting for a slow input's barrier to arrive. Rising alignment time is backpressure and checkpointing cost meeting each other exactly where 10.5 said they would.

None of these four is useful read alone — a job can have perfectly normal lag while one task quietly backpressures the rest, or a healthy backpressure reading while state size climbs toward a leak. They're a set, watched together.

In practice, the **last three come from the engine itself** rather than anything custom-built: Flink exposes per-operator and per-subtask metrics — checkpoint history with trigger time and state size, a per-task backpressure sampler, bytes buffered during alignment — through its own Web UI and a REST API built for exactly this purpose. What's genuinely custom is where those numbers *go*: engine-native metrics are typically exported into a general-purpose monitoring stack — Prometheus scraping the metrics, Grafana for dashboards and alerting — so all four signals sit next to whatever else a team already watches.

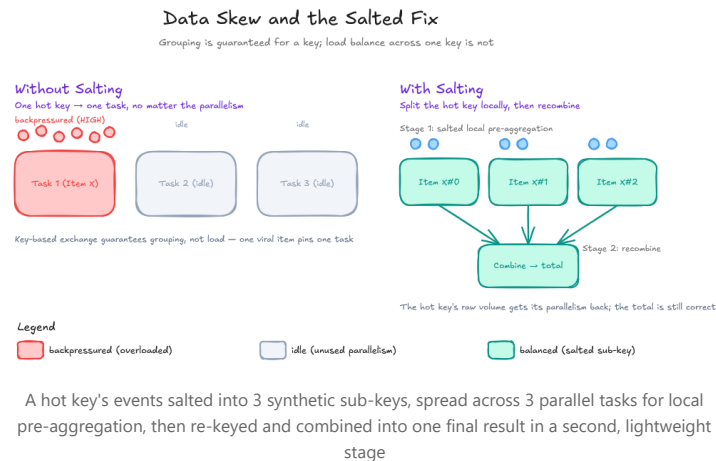
13.2 Skew and Hot Keys

Here's where Section 3.4's key-based exchange guarantee — every event for a given key lands on the same task, always — turns from a correctness property into a worry during production operations. It says nothing about *load*, only about *grouping*. If one catalog item goes viral, or one ad campaign dominates a day's traffic, every one of that key's events lands on the same single task no matter how much parallelism the operator has elsewhere — ten parallel instances of the aggregation operator don't help a key confined to one of them. In practice this shows up first as exactly 13.1's backpressure signal: one subtask pinned at HIGH, the rest sitting comfortably idle.

The generic fix for uneven load — a plain rebalance, redistributing events evenly regardless of key — is the wrong tool here, and worth ruling out explicitly rather than reaching for by instinct: redistributing a keyed aggregation's input randomly breaks the very guarantee that made the aggregation correct in the first place, scattering one key's events across tasks that can no longer produce one right answer for it.

The actual fix works *with* keying instead of around it: a **two-phase, local-then-global aggregation**. Temporarily **salt the hot key with a small**

random suffix — spreading one catalog item's clicks across several synthetic sub-keys — and run the first aggregation stage keyed on that salted key, so the load genuinely spreads across multiple tasks, each holding a partial count. A second, much smaller aggregation stage then re-keys on the *original* key and combines just a handful of partial results per key — not the full raw volume — into the true total. The hot key's raw volume gets its parallelism back; the final answer is still correct, because nothing about grouping was compromised, only the number of stages it took to get there.



13.3 The Always-On Cost

Section 2 named this honestly from the start and it's worth closing the loop on here: a streaming job's cluster footprint is a **standing cost, paid continuously**, whether traffic is heavy at noon or silent at 3 a.m. — not a per-run cost like a nightly batch job that finishes and releases everything it borrowed. That's the real price of the low-latency answer this entire article has built toward, and the four signals in 13.1 are what actually justify spending it: a **job that's healthy on all four is buying genuine freshness** with that always-on cost. A job quietly leaking state or backpressured on one hot key is paying the same 24/7 bill for a pipeline that's already failing to deliver what it's costing.

14. Keeping Derived Systems Current

14.1 The Debt This Section Clears

8B closed on a promise: the **machinery that keeps a search index, a recommendation model, or a geo-index of moving drivers genuinely current**, rather than periodically refreshed. 8A's near-real-time refresh interval was the best answer available at the time, and it was always a compromise — poll the source more often, and you get *less stale*, never *not stale*. This section is where that compromise gets replaced with an actual architecture.

The running example writes itself: the **LMS's own catalog search index** — the Solr/Elasticsearch system 8A built — currently refreshes on that same interval. A newly added title, or an availability count that just changed because someone returned a book, sits invisible to search for however long that interval is. Everything below is how that gap actually closes.

14.2 CDC → Stream Processor → Index

The shape is a straight line: a database's changes are captured as

a **changelog** — Debezium, Part 7D's territory for the connector mechanics — and that changelog becomes the *input stream* to a stream processing job, which transforms it and writes the result into whatever derived system needs to stay current. For the catalog example: an update to the `catalog_items` table's availability count is **captured as a change event**, picked up by a stream processor that reshapes it into whatever document format the search index expects, and written straight into that index — no polling, no interval, no scheduled refresh at all. The moment the source database commits, the pipeline is already moving.

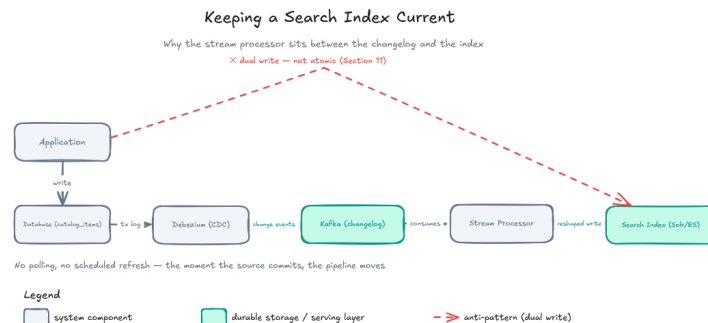
This is also precisely the stream-table duality Section 8.5 introduced: the changelog CDC produces is the versioned-table concept from that section, made concrete and flowing continuously rather than described in the abstract.

14.3 Why the Processor Sits in Between

The tempting shortcut is having the application write directly to both the primary database and the search index in the same request, skipping the stream processor entirely. Two reasons that shortcut doesn't hold up:

It's not actually atomic. Writing to two independent systems in one request has exactly the failure mode Section 11 spent an entire section on: a crash between the two writes leaves them inconsistent, with no transaction spanning both to roll either one back. Section 11.4's Iceberg sink solved this for one destination using checkpoint-coordinated commits; a stream processor sitting between the source and the index gets that same exactly-once machinery essentially for free, applied to a *different* destination, rather than needing to be hand-rolled again in application code.

The transformation itself is real work that doesn't belong in the write path. A search index's document is rarely a 1:1 copy of a database row — it's often assembled from several tables, joined and denormalized into the shape a query engine actually wants. Doing that inline, in the same request that writes the primary record, couples the application's write path to the index's schema — and to every *other* derived system's schema, the moment a recommendation model or a geo-index needs the same underlying change. A stream processor decouples the source of truth entirely from however many derived systems consume its changes: adding a new one means adding a new consumer of the changelog, not touching the original write path at all.



Keeping a Search Index current with each change in the Application, applying CDC with Debezium, capturing changelog in Kafka and reshaping the write in the Stream Processor before writing to the derived system

14.4 Queryable State, Honestly

A reasonable question at this point: if the stream processor already holds state relevant to a query — why not just query the job directly, skip the derived system? Worth being straightforward about where this actually

derived system. Storm being straightforward about where the actualy stands rather than presenting it as a live option: Flink's own queryable state feature exists but is deprecated. Kafka Streams' interactive queries are real and more actively maintained, but scoped to that one framework — 9B's territory once it's actually on the table. Storm's DRPC answered a version of this question but predates the frameworks most teams reach for today.

The honest, general answer, almost always, is: you **don't query the stream processor's live state directly**. You write the state's relevant projection out to a system built for serving queries — exactly Part 7D's Pinot, Druid, or a search index like the one this section's example just fixed — and query that instead. This isn't a limitation to work around; it's the same division of labor this whole section just built: the stream processor's job is to keep the derived system current, not to *be* the derived system.

15. Event-Driven Microservices vs. Stream Processors

15.1 Same Substrate, Different Job

Both of these consume from Kafka. Both react to events rather than being polled or called directly. That surface resemblance is exactly why the distinction is worth making explicit rather than assumed — because underneath it, they're solving genuinely different problems.

Part 3's Payment Service is the clean example of the first kind: it consumes an order event, runs business logic — charge the card, reserve the amount — writes the result to its own database using the transactional read-process-write pattern, and produces a payment event. That's the entire computation. The next order that arrives is handled completely **independently**; nothing about how order #4471 was processed carries forward to affect order #4472. The event is a **self-contained unit of work** — received, processed, done.

Our own project's session aggregation is the opposite shape entirely. A single click event, on its own, means almost nothing — its significance depends entirely on what other events belong to the same member's session, what window it falls into (Section 7), whether a matching borrow shows up within the interval join's bound (Section 8.2). This is the real distinction worth holding onto: **an event-driven microservice reacts to one event at a time, with business logic and its own database; a stream processor is a continuous dataflow with managed state and explicit time semantics spanning many events**. Everything from Section 6 onward — watermarks, windows, joins, checkpointed state — exists specifically because a stream processor's answer depends on more than the one record currently in hand.

	Event-Driven Microservice	Stream Processor
Unit of work	One event, processed and completed independently	An unbounded sequence, computed over continuously
State	Its own database, scoped to entities it owns	Managed operator state (Section 9), scoped to keys and windows
Time awareness	None inherent — "now" is whenever the event arrived	Explicit event-time semantics, watermarks, windows
Failure model	Local transaction, at-least-once delivery, idempotent consumer	Checkpoint-coordinated distributed snapshot (Section 10)

Contrasting an Event-Driven Microservice to a Stream Processor

15.2 Where the Line Blurs — and the Outbox Pattern

The boundary gets genuinely fuzzy at the edges, and it's worth naming rather than pretending these are always crisp categories. Kafka Streams (9B) is a *library*, not a cluster — it runs embedded inside an ordinary application process, which means an otherwise unremarkable microservice can absolutely contain real stream-processing logic inside it: a KTable-based enrichment join (Section 8.5) living quietly inside what looks, from the outside, like any other Spring Boot service. What actually determines which category a piece of logic belongs to isn't the codebase or the deployment shape — it's **whether it needs the scaffolding** Sections 6 through 10 built: time semantics, cross-event state, checkpointed recovery. A service that reacts to one event and writes one database row doesn't need any of that, regardless of which library happens to be sitting inside it.

This is exactly where Part 3's **outbox pattern** turns out to be the bridge between these two worlds, not a detail specific to microservices alone. The problem it solves — atomically writing a business record and reliably publishing the event describing it, without a distributed transaction — is solved by writing both the business row and the event into the *same local database transaction*, with a separate process (or a CDC tool like Debezium, Section 14's exact mechanism) reading the outbox table and publishing to Kafka. An event-driven microservice's job ends the moment that event is durably on the log. A stream processor's job begins there: everything from that point forward — the continuous, stateful, time-aware computation over the events the microservice layer produced — is this article's territory, not that one's.

Part 5B's Loan Service, emitting a LoanProcessed event through its CQRS write path, is a microservice in exactly this sense: one loan, one event, no computation spanning loans. The catalog-browse clickstream's session aggregation, consuming from that same kind of durable event backbone, is a stream processor. Same substrate underneath both. Genuinely different computational model on top.

16. The LMS Through-Line & Hands-On Project

16.1 What We're Building, and What We're Not

Everything from Section 3 onward has described mechanisms as they exist in engines built by large teams over a decade. The fastest way to genuinely internalize those mechanisms is to build a small one yourself — not to compete with Flink, but to be forced into making, explicitly and in code, the same design decisions Flink's authors made, and to feel why each one is there.

So the honest framing up front: this is a **teaching engine**. It will run real user-defined operators across real, separate worker processes, genuinely checkpoint state, genuinely recover from a killed worker. It will not have a query optimizer, a SQL layer, dynamic rescaling, or any of the production hardening that separates "demonstrates the mechanism" from "trust this with your revenue." Where we deliberately take a shortcut, the article says so out loud — the same way 8A's Solr demo was explicit about running a single ZooKeeper instance rather than quietly implying a production setup.

What that buys is specific: when 9B walks through Flink's JobManager or Storm's Nimbus, you'll be reading about a component whose job you've already had to do yourself.

16.2 The Use Case, Before the Design

Good system design starts from what the business actually needs, not from a component diagram — so before a single process gets designed, here's the concrete problem.

The LMS has a catalog-browse clickstream: members searching the catalog, clicking search results, and sometimes borrowing what they clicked. Today, whoever wants to understand that behavior queries the warehouse built in Part 7A — which means every question they can ask is a question about yesterday. The three things the library's collections team actually wants are:

1. **Which search terms produce clicks but no borrows** — a signal that the catalog metadata is misleading, or the item was unavailable when they got there.
2. **How long members actually spend browsing before borrowing** — session-shaped behavior, not row-shaped.
3. **Which newly-added titles are getting attention right now** — a question whose entire value evaporates if the answer is 24 hours old.

Notice what those three demand, mapped against the systems this series has already built. A relational database (Parts 2, 4A) can store the events, but answering "sessions per member with a 15-minute inactivity gap" means a query that re-scans and re-groups the whole day's events on every run. A warehouse or Spark batch job (Part 7) computes it correctly and efficiently — once per scheduled run, which is precisely the gap Section 1 opened on. What's genuinely different about this class of problem is that the computation is **incremental and continuous** — each new click updates an answer that already exists, rather than triggering a recomputation from scratch — and that's the computational model this whole article has been describing.

16.3 Requirements, Functional and Non-Functional

Functional. The engine must accept a user-defined job as a DAG of operators (Section 3.1), distribute its tasks across multiple worker processes, and support four operator types spanning the article's core mechanisms:

Operator	Mechanism it exercises
Stateless filter	Basic task execution, forward exchange (Section 3.4)
Windowed aggregation, keyed by member	Keyed state (Section 9.1), event-time session windows (Section 7.1), watermarks (Section 6.2)
Stream-stream interval join (click → borrow)	Two-sided buffered state, watermark-driven eviction (Section 8.2-8.3)
Iceberg sink	Checkpoint-coordinated transactional commit (Section 11.4)

The Operator types we plan to support in our Distributed Stream Processing Engine Design

Non-functional — the three that actually shape the architecture:

- **Correctness under failure.** A worker dying mid-window must not produce a double-counted or missing result. This is the requirement that forces checkpointing (Section 10) and a transactional sink (Section 11) into the design — not as features, but as the only way to meet the requirement at all.
- **Freshness.** Results visible within seconds of the event, not hours. This is what rules out any design where the sink batches up and flushes on a schedule, and what makes checkpoint interval

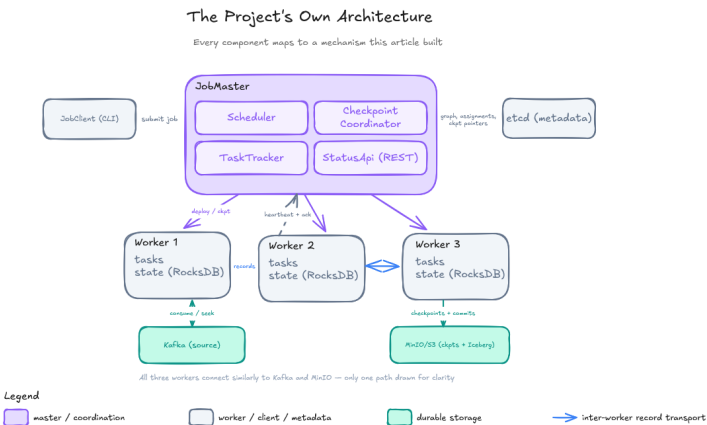
a *product* decision, not just an operational one: with a transactional sink, checkpoint interval directly sets result visibility latency (Section 11.3).

- **Observable job state.** At any moment, it must be answerable from outside the cluster: is this job running, which tasks are alive, when did the last checkpoint complete. Section 13's signals, made real.

16.4 High-Level Architecture

The component list falls out of the requirements above almost mechanically, which is rather the point:

- **A master process** holding the submitted job graph, converting it to a physical graph, assigning tasks to workers, tracking task liveness via heartbeat, and coordinating checkpoints (Sections 4.1, 4.3, 10.2).
- **N worker processes**, each running assigned tasks in threads, holding local state, and exchanging records with other workers (Sections 4.2, 5.2).
- **A checkpoint coordinator** inside the master, injecting barriers at the sources and tracking per-task acknowledgments until a checkpoint is complete (Section 10.4).
- **A state store per task**, embedded and local, with checkpoints written to shared durable storage (Section 9.3).
- **A status API** on the master exposing job state, task state, and checkpoint history (Section 13.1).



Project Architecture - Client submitting a Job to the Master, Master's Scheduler and Checkpoint Coordinator, 3 Worker Processes each running Tasks with local State, record exchange between Workers, Checkpoint writes to durable storage, status API

16.5 What We Build vs. What We Borrow

Build ourselves	Borrow	Why
Job graph, scheduler, task assignment	—	This is the lesson
Checkpoint coordination and barrier alignment	—	The article's centerpiece mechanism (Section 10.3)
Watermark generation and propagation	—	Section 6.3's minimum-across-inputs rule, implemented
Operator state and its checkpointing	Embedded KV store (RocksDB or equivalent)	Section 9.3's backend choice, not its internals
—	gRPC for control plane and record transport	Section 5.2's mechanism understood; hand-rolling a network stack teaches nothing new
—	Kafka as the resettable source	Section 11.1's prerequisite; Part 3 already covered it

	source	already covered
—	etected or ZooKeeper for master metadata	Section 4.5's honest answer; hand-rolled consensus is Part 6's topic, not this one

The boundaries for the Components we build vs borrow

The last row is worth being loud about: our master is a single point of failure by default, exactly as Flink's JobManager is (Section 4.5). Storing its metadata externally is what makes recovery *possible*; a full standby-with-leader-election setup is noted as the production answer and left as an extension rather than pretended into existence.

16.6 The Demos — Where the Article Gets Proven

Each demo maps to a claim the article made, and exists to make that claim falsifiable rather than merely stated:

1. **Kill a worker mid-window.** The job recovers from the last checkpoint, reprocesses the events after it, and the final session count is correct — no double-count, no gap. Proves Sections 10.6 and 11.
2. **Kill the master.** Workers keep processing; on master restart, job metadata is recovered from external storage and task tracking resumes. Proves Section 4.5's distinction between "lost coordination" and "lost work."
3. **Feed a late event.** An event arriving after its window's watermark has passed lands in the side output; an event within allowed lateness triggers an updated result, and the Iceberg sink overwrites rather than duplicates. Proves Sections 7.4 and 11.2.
4. **Hammer one key.** Traffic concentrated on a single catalog item drives one task's state and checkpoint duration up while its siblings stay idle; applying the salted two-phase aggregation flattens it. Proves Sections 13.2 and 9.4.

That fourth demo is the one that tends to change how people think, because it fails in a way that looks like nothing is wrong — every dashboard green, no errors, throughput fine, one task quietly drowning.

16.7 Build Phases and Handoff

The build is phased so each stage runs and demonstrates something before the next begins: (1) single-process engine running a DAG of operators against a Kafka source, (2) split into master and worker processes with gRPC and real task assignment, (3) keyed state and event-time session windows, (4) checkpointing with barrier alignment and recovery, (5) the interval join, (6) the transactional Iceberg sink, (7) the status API and the four demos.

As with every project in this series, the full phased implementation guide ships as a companion PDF and a CLAUDE.md handoff file in the GitHub repository, structured so the build can be driven iteratively with a code-generation tool rather than typed out by hand — the article's job is the design reasoning; the repository's job is the working code.

Project Companion [PDF](#)

GitHub repository link: [Part-9A-Project-Building-a-Distributed-Stream-Processing-Engine](#)

17. Takeaways

The fundamentals, in the order this article built them:

- **A stream processing job is a DAG, compiled twice** — once by you (the logical graph), once by the engine (the physical graph of parallel tasks) — and everything from data serialization to exchange strategies to task chaining exists to move records across that physical graph as cheaply as the topology allows.
- **Master and worker are universal roles**, however differently two engines name them — and where a slot's granularity differs (Storm's whole-JVM vs. Flink's pipeline-slice).
- **Time is a choice, not a given**. Event time buys reproducible results on replay; watermarks are the engine's only way of knowing when it's safe to act on an event-time claim.
- **A checkpoint is a distributed snapshot taken without stopping the stream** — barrier alignment is the mechanism, and it's the same mechanism a transactional sink piggybacks on to get exactly-once output.
- **State is the thing that makes streaming genuinely harder than batch** — not per-record overhead, which chaining and batching already claw back, but a permanent operational surface that has to be checkpointed, bounded, and rescaled without loss.
- **Correctness has a size** — an unbounded join, a leaking key, an unbounded window are all the same failure shape: no principled moment ever arrives to say "this can be forgotten."
- **Reprocessing is a first-class capability, not an emergency procedure**. A resettable source, event time, and savepoints together are what turn "replay the log" from an idea into something you can actually do, safely, whenever you need to.
- **The stream processor's job is to keep something else current** — a derived index, a table, a report — not to be the thing queried directly. That division of labor is what actually closes 8B's promise.

If one idea deserves to outlast the rest of this article: almost every mechanism here exists because a stream never closes. Batch systems get to treat "the whole dataset" as a stable, known thing to reason about. A stream processor never gets that luxury — and everything from watermarks to checkpoints to savepoints is the engineering answer to operating in those circumstances.

9B picks this up in the frameworks that actually run in production — Storm, Flink, Spark Structured Streaming, Kafka Streams — each one a different, opinionated answer to the exact same set of problems this article just named.

18. Deep Dives

Three Deep Dives carry this article's implementation-level detail forward — full mechanics rather than summaries, built as complete standalone documents once the main content is locked.

DD1 — RocksDB as a State Backend (*extends Section 9.3*) The concept-level treatment in the main article — an embedded, per-task LSM store, chosen because disk-backed state isn't capped by JVM heap — gets its full mechanics here: the write path through memtable and write-ahead log

mechanics here: the write path through meritable and write-ahead log, how column families isolate each declared state name from every other, and the detail the main article deliberately held back for this document — why SST file immutability is what makes incremental checkpointing possible, uploading only the files that changed rather than the whole state store on every checkpoint. Closes with practical tuning: block cache and write buffer sizing, at the level of what to know rather than an exhaustive reference. [\[PDF\]](#)

DD2 — The Checkpointing Algorithm in Full (*extends Section 10*) The barrier-alignment mechanics from Section 10.3, worked through with more cases than the main article had room for — multi-input, multi-output, and the interaction with backpressure from Section 10.5 in full detail. This is also where both honesty flags from Section 10 get resolved with real source material once available: true unaligned checkpoints (persisting in-flight buffered records rather than downgrading to at-least-once), and fine-grained, region-scoped recovery as the refinement beyond the whole-job restart Section 10.6 described as the baseline. [\[PDF\]](#)

DD3 — Exactly-Once Sinks in Detail (*extends Section 11*) Full implementation mechanics for both sink strategies Section 11.3 introduced at concept level: what a write-ahead-log sink actually buffers and when, and precisely what a sink system has to expose to support genuine two-phase commit. The centerpiece is the Iceberg commit protocol in full — manifest lists, snapshot metadata, and what happens when two commits race — going well past Section 11.4's "atomic snapshot swap" summary. [\[PDF\]](#)

19. References

Books

- *Storm Applied: Strategies for Real-Time Event Processing* — Sean T. Allen, Matthew Jankowski, Peter Pathirana (Manning). Source for Storm's architecture (Nimbus, Supervisor, worker processes), the spout/bolt model, the tuple tree and acking, and the executor-level internals — disruptor queues, local vs. remote transfer — covered in Sections 4 and 5.
- *Stream Processing with Apache Flink* — Fabian Hueske, Vasiliki Kalavri (O'Reilly). The primary source across most of this article: dataflow programming and task chaining (Section 3), data serialization's type-system rationale, time and windowing (Sections 6–7), joins (Section 8), state and RocksDB (Section 9), checkpointing, barrier alignment, and recovery (Section 10), delivery semantics and sink connectors (Section 11), and savepoints (Section 12).

Papers

- Carbone, P., Katsifodimos, A., Ewen, S., Markl, V., Haridi, S., Tzoumas, K. (2015). *Apache Flink: Stream and Batch Processing in a Single Engine*. IEEE Data Engineering Bulletin. Source for the dataflow graph model and Figure 2's process architecture (Section 4), the buffer-timeout latency/throughput benchmark (Section 2), Asynchronous Barrier Snapshotting and its Chandy-Lamport lineage (Section 10), and the Lambda architecture critique that opened Section 12's Kappa framing.
- Akidau, T., et al. (2015). *The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing*. VLDB. Added to the

source pile during Section 7's drafting; not yet directly cited in this article's current text, but the authoritative source for the what/where/when/how framing and the discarding/accumulating/accumulating-and-retracting accumulation modes — planned for DD3 once incorporated properly.

Primary Documentation

- Apache Flink documentation — *Flink Architecture*, <https://nightlies.apache.org/flink/flink-docs-stable/docs/concepts/flink-architecture/>. The canonical Client/JobManager/TaskManager diagram, cited alongside the 2015 paper's Figure 2 for Section 4's daemon architecture diagram.
- Apache Storm documentation - <https://storm.apache.org/releases/3.0.0/index.html>
- LMAX Disruptor — official site, <https://lmax-exchange.github.io/disruptor/>, and source repository, <https://github.com/LMAX-Exchange/disruptor>. Primary sources for Section 5's treatment of the disruptor queue underlying Storm's executor transport.
- Debezium Architecture - <https://debezium.io/documentation/reference/stable/architecture.html>

Comments

 7

 ▾

 Like

 Comment

 Share

Add a comment...



No comments, yet.

Be the first to comment.

(Start the conversation)